

SSN College of Engineering, Kalavakkam
Department of Computer Science and Engineering
V Semester - CSE 'A'
UCS2511 NETWORKS LAB
Exercise 6
MultiUserChat

Name : Aathish Sivasubrahmanian 3122215001001

Aim:

Chat in C.

server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <fcntl.h>
#include <pthread.h>
#include <netinet/in.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <arpa/inet.h>
```

```
int clientCount = 0;
```

```
static pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
static pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
```

```
struct client{
```

```
    int index;
    int sockID;
    struct sockaddr_in clientAddr;
    int len;
```

```

};

struct client Client[1024];
pthread_t thread[1024];

void * doNetworking(void * ClientDetail){

    struct client* clientDetail = (struct client*)
ClientDetail;

    int index = clientDetail -> index;
    int clientSocket = clientDetail -> sockID;

    printf("Client %d connected.\n",index + 1);
    while(1){

        char data[1024];
        int read = recv(clientSocket,data,1024,0);
        data[read] = '\0';

        char output[1024];
        if(strcmp(data,"LIST") == 0){

            int l = 0;
            for(int i = 0 ; i < clientCount ; i ++){

                if(i != index)
                    l += snprintf(output +

l,1024,"Client %d is at socket %d.\n",i + 1,Client[i].sockID);
            }

            send(clientSocket,output,1024,0);
            continue;

        }
    }
}

```

```
if(strcmp(data,"SEND") == 0){
```

```
    read = recv(clientSocket,data,1024,0);  
    data[read] = '\0';
```

```
int id = atoi(data) - 1;
```

```
read = recv(clientSocket,data,1024,0);  
data[read] = '\0';
```

```
    send(Client[id].sockID,data,1024,0);  
}
```

```
}
```

```
return NULL;  
}
```

```
int main(){  
    int serverSocket = socket(PF_INET, SOCK_STREAM, 0);  
    struct sockaddr_in serverAddr;
```

```
    serverAddr.sin_family = AF_INET;  
    serverAddr.sin_port = htons(8080);  
    serverAddr.sin_addr.s_addr = htons(INADDR_ANY);
```

```
    if(bind(serverSocket,(struct sockaddr *) &serverAddr ,
```

```

sizeof(serverAddr)) == -1) return 0;

    if(listen(serverSocket,1024) == -1) return 0;

    printf("Server started listenting on port
8080 .....\\n");

while(1){

    Client[clientCount].sockID = accept(serverSocket,
(struct sockaddr*) &Client[clientCount].clientAddr,
&Client[clientCount].len);

    Client[clientCount].index = clientCount;

    pthread_create(&thread[clientCount], NULL,
doNetworking, (void *) &Client[clientCount]);

    clientCount ++;
}

for(int i = 0 ; i < clientCount ; i ++)
    pthread_join(thread[i],NULL);

}

```

client.c

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>

```

```
#include <fcntl.h>
#include <pthread.h>
#include <netinet/in.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <arpa/inet.h>
```

```
void * doRecieving(void * sockID){
    int clientSocket = *((int *) sockID);
    while(1){
```

```
        char data[1024];
```

```
        int read = recv(clientSocket,data,1024,0);
        data[read] = '\0';
        printf("%s\n",data);
```

```
    } }
```

```
int main(){
    int clientSocket = socket(PF_INET, SOCK_STREAM, 0);
    struct sockaddr_in serverAddr;
```

```
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(8080);
    serverAddr.sin_addr.s_addr = htonl(INADDR_ANY);
```

```
    if(connect(clientSocket, (struct sockaddr*) &serverAddr,
sizeof(serverAddr)) == -1) return 0;
```

```
printf("Connection established .....\\n");
```

```
pthread_t thread;
```

```
pthread_create(&thread, NULL, doRecieving, (void *)  
&clientSocket );
```

```
while(1){
```

```
    char input[1024];  
    scanf("%s",input);
```

```
    if(strcmp(input,"LIST") == 0){  
        send(clientSocket,input,1024,0);
```

```
}
```

```
if(strcmp(input,"SEND") == 0){  
    send(clientSocket,input,1024,0);
```

```
    scanf("%s",input);  
    send(clientSocket,input,1024,0);
```

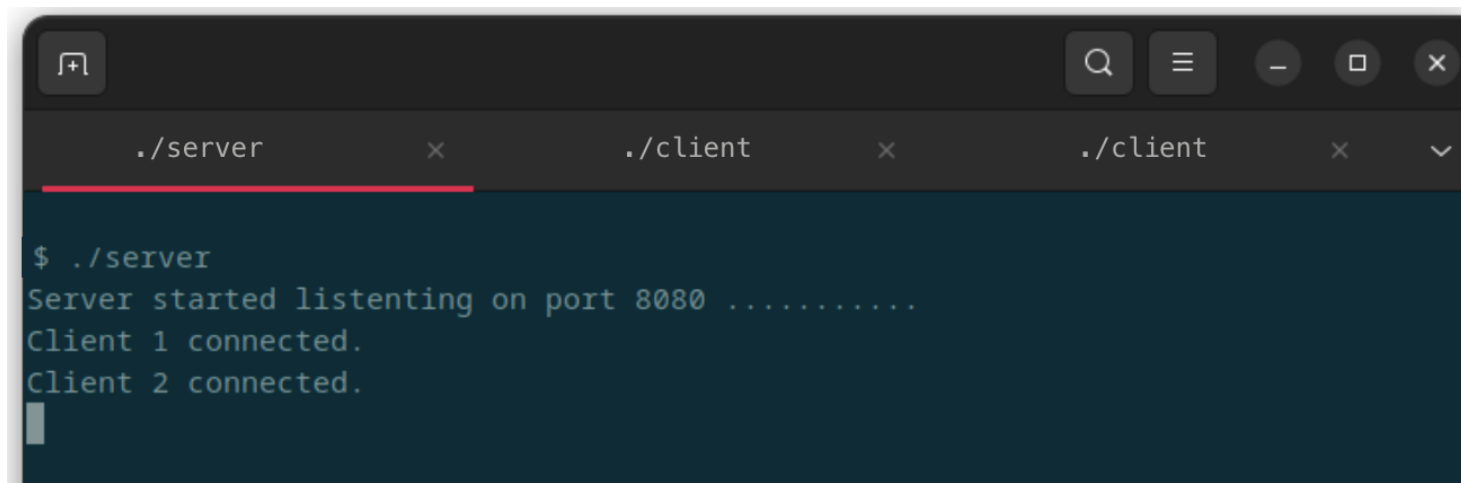
```
    scanf("%[^\n]s",input);
```

```
    send(clientSocket,input,1024,0);
```

```
}
```

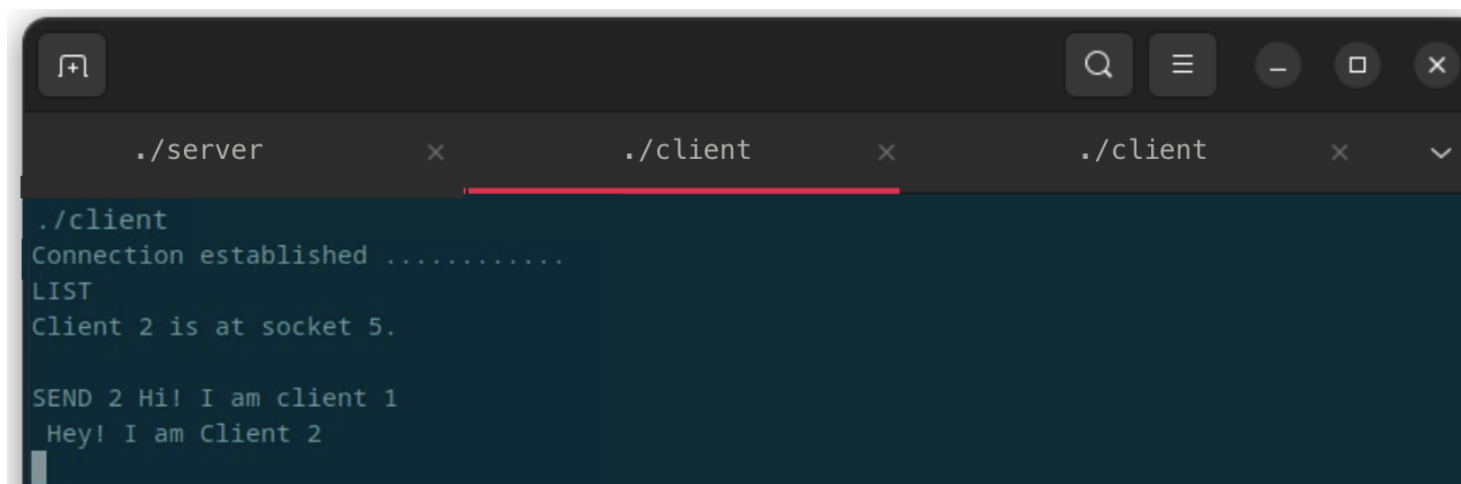
```
}}
```

O/P:



A terminal window with three tabs: `./server`, `./client`, and `./client`. The `./server` tab is active and shows the following output:

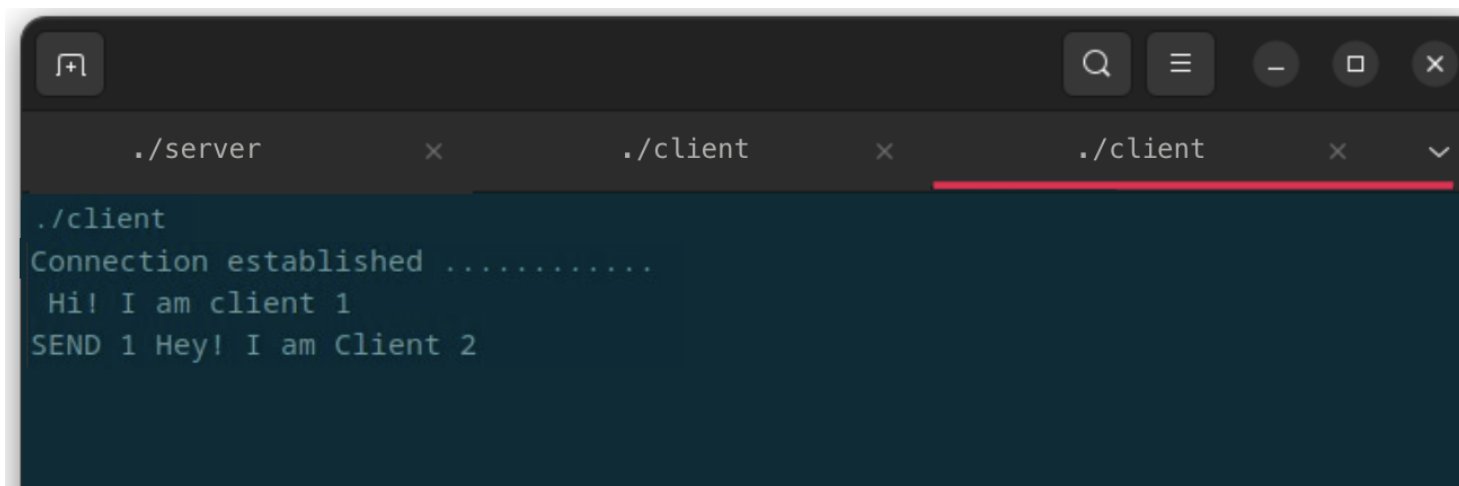
```
$ ./server
Server started listenting on port 8080 .....
Client 1 connected.
Client 2 connected.
```



A terminal window with three tabs: `./server`, `./client`, and `./client`. The first `./client` tab is active and shows the following output:

```
./client
Connection established .....
LIST
Client 2 is at socket 5.

SEND 2 Hi! I am client 1
Hey! I am Client 2
```

A terminal window with a dark background and light-colored text. The window has a title bar with standard macOS window controls (red, yellow, green buttons) and a search icon. Below the title bar, there are three tabs: './server', './client', and './client'. The third tab is selected and highlighted with a red underline. The terminal content shows the following text:

```
./client
Connection established .....
Hi! I am client 1
SEND 1 Hey! I am Client 2
```

Learning Outcomes:

I learned how to simulate ARP using socket programming.